ST ALOYSIUS
(DEEMED TO BE UNIVERSITY)
MANGALURU 575 003 - INDIA
SCHOOL OF ENGINEERING

Multi-Process Microcontroller Simulator: Design, Implementation & Evaluation of STC89C52 Microcontroller

Register Number	Student Name
25190111	CYRUS SHOBITH PINTO
25190110	CHRISEL LOBO
25190106	ARNOLD NOAH KARYAPPA
25190148	SHASHIDHARA POOJARI

Department of Computer Science & Engineering

Project-Based Learning (PBL) Report

ABSTRACT

This project presents the design, implementation, and empirical evaluation of an educational, instruction-accurate multi-process software simulator for an 8051-subset (STC89C52) 8-bit microcontroller [1], [2]. Traditional educational microcontroller simulators frequently rely on single-threaded monolithic architectures, where user interface latency or synchronous disk file I/O can stall the simulated CPU core. To eliminate this coupling, we engineered a decoupled three-process architecture communicating over Java TCP loopback stream sockets: CoreProcess (17-opcode 8051 CPU engine, 8 KB code memory, 256-byte internal RAM, 16-slot FIFO queue, and non-blocking bounded 1,024-slot ArrayBlockingQueue log dispatcher on port 5000), LoggerProcess (thread-safe append-only execution audit daemon writing to simulator.log on port 5001), and UIProcess (interactive HTTP web server and /api/cmd control endpoint on port 8080). The CPU core models the Fetch-Decode-Execute pipeline across 17 representative machine opcodes (including both synthetic 0xFF HALT and standard 8051 SJMP $ [80 FE] self-loop halting), general-purpose registers R0-R7, hardware PSW flags (CY, AC, OV, P), and stack pointer (SP initialized at 07H) [2], [3]. Furthermore, we implemented the 3GPP TS 36.212 CRC-24b (g(D) = 0x800063) error detection algorithm [5] via step8051Hardware(), shifting bit-by-bit through the carry flag across 8-bit registers R5, R6, and R7. Benchmarks on OpenJDK 26.0.2 (Windows 11 x64, 200-iteration JIT warm-up) recorded ~1.0 ms for 1,000 in-memory CRC-24b computations (0.001 ms/pkt) versus 0.666 ms average round-trip latency over stop-and-wait TCP loopback IPC (~1,500 msgs/sec), while fault-injection burst testing verified that a 1,500-event log flood completes in 11.30 ms with 476 graceful queue-overflow drops and zero CPU stalls across all 34 automated test assertions.

Keywords: Microcontroller Emulation, Multi-Process Architecture, Inter-Process Communication, Fetch-Decode-Execute, Instruction-Accurate Virtualization, CRC-24b Error Detection.

TABLE OF CONTENTS

1. Introduction	4
1.1 Problem Statement	4
1.2 Project Objectives	4
2. The Simulator Design and Working	4
2.1 Microcontroller Architecture Study	4
2.2 System Architecture and Design Decisions	4
2.3 Operational Workflow and GUI Layout	4
2.4 Incremental Implementation by Week	5
2.5 Multi-Process IPC Architecture & Benchmarking	5
3. Simulation Examples Testing and Screenshots	5
4. Individual Contribution	6
4.1 Weekly Task Breakdown by Member	6
4.2 Agreed Team Contribution Summary	6
4.3 Individual Personal Reflections	6
5. Team Communication	7
6. Problems Faced and How They Were Resolved	7
7. From Theory to Practice	7
7.1 Operating Systems Concepts	7
7.2 Data Structures & Algorithms	7
7.3 Computer Networks	7
7.4 Cross-Disciplinary Synthesis Reflection	7
8. Conclusion and Future Work	8
References	8
Final Report Sign-off & Faculty Mentor Endorsement	9

1. Introduction

1.1 Problem Statement

Modern computer engineering pedagogy requires students to understand how hardware state machines, memory segmentation, and instruction pipelines execute in silicon [1], [4]. However, physical hardware kits present maintenance bottlenecks and lack visibility into internal register banks and pipeline stages. Furthermore, existing educational simulators [3] are predominantly monolithic, single-threaded applications where GUI event handling or synchronous disk logging can stall the simulated CPU loop. Additionally, executing telecommunications error-detection algorithms—such as the 3GPP LTE 24-bit cyclic redundancy check (CRC-24b) [5]—across an 8-bit microcontroller register file requires multi-byte register cascading (R5:R6:R7) and bit-level shifting through the carry flag in GF(2) arithmetic.

1.2 Project Objectives

1. Virtualize a 17-opcode educational subset of the STC89C52/8051 microcontroller [2], [3] in Java with instruction-accurate Fetch-Decode-Execute stages, general-purpose registers R0–R7, and hardware PSW flags (CY, AC, OV, P).
2. Implement an OS-level multi-process architecture over Java TCP loopback sockets (ports 5000, 5001, 8080) with a bounded non-blocking ArrayBlockingQueue(1024) log dispatcher.
3. Construct a responsive Web Dashboard (Port 8080) featuring real-time mnemonic disassembly, hardware stack visualization, and 16-slot circular FIFO queue inspection.
4. Build an independent, thread-safe append-only audit logging daemon on Port 5001 to persist execution traces into simulator.log without stalling CPU execution.
5. Implement and benchmark the 3GPP TS 36.212 CRC-24b algorithm [5] via 8-bit register shifting across R5:R6:R7, demonstrating clean zero-syndrome checks (0x000000) and deterministic single/double-bit noise detection.

2. The Simulator Design and Working

2.1 Microcontroller Architecture Study

The target MCU (STC89C52RC / Intel MCS-51 [2]) possesses an architectural hierarchy mapped within our 17-opcode educational virtualization environment as shown in Table 1.

Table 1: *STC89C52 / MCS-51 Architectural Specification and Virtualization Treatment [2]*

Hardware Feature	Architectural Implementation Specification	Virtualization Treatment
Data Word Size	8-bit ALU & Datapath	Constrained with bitwise mask (& 0xFF) across all registers and memory cells to prevent Java 32-bit sign-extension.
Program Counter (PC)	16-bit addressing (8 KB On-Chip ROM)	16-bit unsigned index into 8,192-byte code memory; halts on synthetic 0xFF or standard 8051 SJMP $ (80 FE).
Internal RAM	256 Bytes on-chip (00H - FFH)	Array partitioned into Register Banks (00H-1FH), Bit-Addressable RAM (20H-2FH), and Scratchpad (30H-7FH).
Special Function Registers	Direct addressing (80H - FFH)	Mapped SFR space: Accumulator (0xE0), B (0xF0), PSW (0xD0), Stack Pointer (0x81), and GPIO Ports P0-P3.
ALU Flags	Program Status Word (PSW)	Discrete boolean flags tracking Carry (CY), Auxiliary Carry (AC), Overflow (OV), and even Parity (P).
Stack & FIFO Queue	Hardware SP + 16-Slot Circular Queue	Pre-increment PUSH / post-decrement POP (SP=07H on reset) plus 16-slot circular FIFO buffer with Front/Rear wrap-around.

2.2 System Architecture and Design Decisions

The simulator executes across three decoupled OS processes over Java TCP loopback stream sockets (Figure 1): CoreProcess (17-opcode CPU engine, 8 KB ROM, 256B RAM, 16-slot FIFO queue, and CRC-24b lab on port 5000), LoggerProcess (thread-safe append-only audit daemon writing to simulator.log on port 5001), and UIProcess (HTTP web server and CLI controller on port 8080). To guarantee that a stalled LoggerProcess never blocks CoreProcess when OS socket buffers fill, CoreProcess enqueues log frames into a bounded non-blocking ArrayBlockingQueue(1024) via offer() with an atomic drop counter drained by a daemon thread.

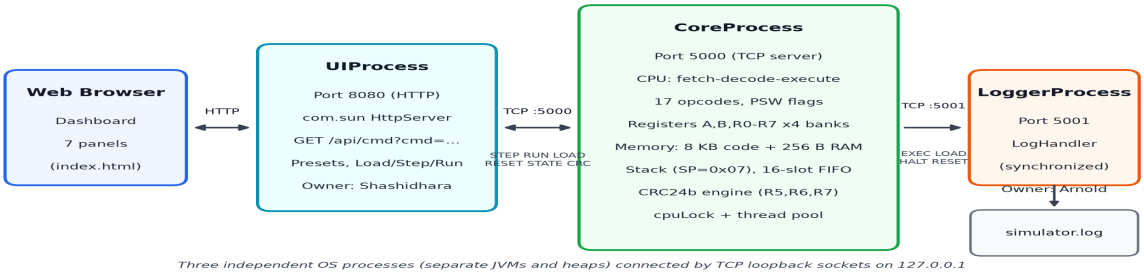


Figure 1: *Three-Process TCP Loopback IPC Architecture (UIProcess:8080 <-> CoreProcess:5000 -> LoggerProcess:5001) with Bounded Non-Blocking Log Queue.*

2.3 Operational Workflow and GUI Layout

The web dashboard on http://localhost:8080/ exposes 7 panels: (1) Program Memory & Live Disassembly with PC pointer, (2) Fetch-Decode-Execute Pipeline Indicator, (3) Working Registers (PC, SP, ACC, B, R0–R7) & PSW Flags, (4) Hardware LIFO Stack Tower, (5) 16-Slot Circular Queue Grid, (6) 16x16 Internal RAM Hex Matrix, and (7) the 3GPP LTE-B CRC-24b Lab. Browser actions invoke GET /api/cmd?cmd=... which UIProcess forwards over TCP Port 5000 to CoreProcess, returning serialized STATE|... frames.

2.4 Incremental Implementation by Week

Table 2: Five-Week Incremental Implementation Schedule and Deliverables [1]

Sprint / Week	Primary Module Focus	Concrete Functional Deliverables
Week 1	Architecture Modeling & Tooling	8051 register model, 8 KB ROM / 256B RAM memory map design, Java project structure, and Git workflow setup.
Week 2	Core ALU & Basic Instruction Set	Fetch-Decode-Execute pipeline, arithmetic/logic opcodes (ADD, SUBB, MUL, DIV, ANL, ORL, XRL), and PSW flags.
Week 3	Data Structures & Memory Virtualization	Hardware stack pointer (SP=07H reset), 16-slot circular FIFO queue, and assembly-level FIFO test (QueueTest.java).
Week 4	Multi-Process Socket IPC & Architecture	Decoupled Core (5000), Logger (5001), UI (8080), TCP loopback sockets, and non-blocking async audit logging.
Week 5	Telecommunications Extension & Benchmarks	3GPP LTE-B CRC-24b engine (0x800063) via step8051Hardware() across R5:R6:R7, noise lab, and BenchmarkCRC.java.

2.5 Multi-Process IPC Architecture & Benchmarking

BenchmarkCRC.java was executed on Windows 11 x64 (OpenJDK 26.0.2) after a 200-iteration JIT warm-up on a 49-byte telecom payload: (1) Standalone in-memory execution completed 1,000 CRC-24b iterations in ~1.0 ms (~0.001 ms/pkt, checksum 0x025A3C); (2) Multi-process stop-and-wait TCP loopback IPC on Port 5000 completed 100 round-trips in 66.6 ms (0.666 ms average RTT/msg, ~1,500 msgs/sec—~660x slower than in-memory calls due to OS kernel socket traversal and 512-byte RAM hex serialization, yet sub-millisecond for interactive stepping); (3) Fault-injection burst testing (LoggerProcessTest TC-06) flooded 1,500 log events against a stalled consumer in 11.30 ms (1,024 queued, 476 dropped via offer()), proving CPU execution never blocks.

3. Simulation Examples Testing and Screenshots

Automated test suites (34/34 passing assertions across TestCPU: 17/17, TestCRC24b: 8/8, QueueTest: 3/3, and LoggerProcessTest: 6/6) validated Data Transfer, Arithmetic/Logic, Control Flow (SJMP & SJMP $), Stack, FIFO Queue, Non-Blocking IPC Backpressure, and 3GPP LTE CRC-24b (Table 3):

Table 3: Automated Verification Matrix Across Instruction Categories, IPC Fault Isolation, and CRC-24b

Test ID	Instruction / Operation	Expected Outcome	Actual Outcome	Status
TC-01	Data Transfer (MOV A,#42H; B,#07H)	A=42H, B=07H, PC advances	A=42H, B=07H; PC incremented	PASS
TC-02	ALU Add/Carry (MOV A,#FEH; ADD #03H)	A=01H, CY=1, OV=0, P=1	A=01H, CY=true, OV=false (TC-04)	PASS
TC-03	Control Flow (SJMP +1 & SJMP $ 80 FE)	Skip INC; 80 FE self-loop halts	A=00H; 80 FE sets isHalt=true (TC-17)	PASS
TC-04	Stack PUSH ACC (0xC0 0xE0, A=3CH)	SP 07H->08H; RAM[08H]=3CH	SP=08H, RAM[08H]=3CH (TC-15)	PASS
TC-05	FIFO Queue (ENQUEUE 0xE0 / DEQUEUE 0xD0)	FIFO order preserved (10, 30, 5)	Dequeued 10, 30, 5; empty=true (3/3)	PASS
TC-06	MUL AB (6x7) & DIV AB (B=0)	A=2AH, OV=0; DIV B=0 sets OV=1	A=2AH, OV=false; DIV0 OV=true (TC-16)	PASS
TC-07	IPC Fault-Isolation Backpressure	1,500 burst <50ms when Logger stalls	11.30 ms; 1,024 queued, 476 dropped	PASS
TC-08	CRC-24b & Noise (Byte 0 Bit 0 / Bits 0,1)	'123456789'->0x23EF52; clean=0	R5:23 R6:EF R7:52; b0=520000, b01=F60000	PASS

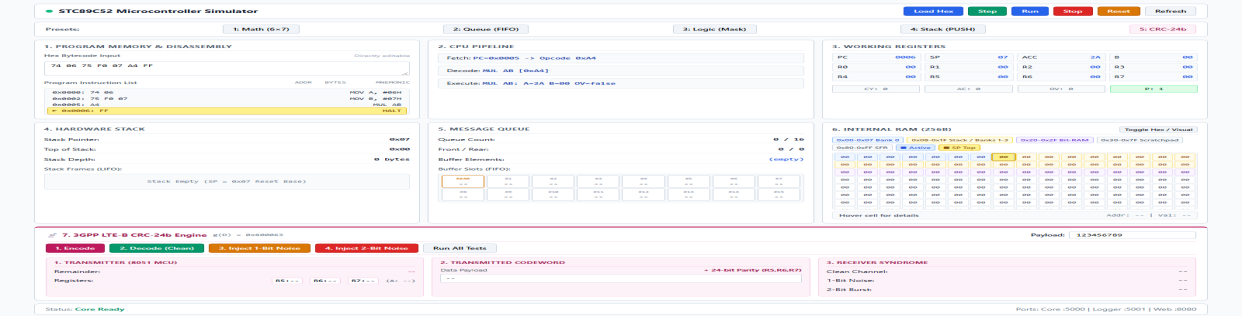


Figure 2: *Live STC89C52 Web Dashboard (http://localhost:8080) after stepping Preset 1 (Math 6x7: 74 06 75 F0 07 A4 FF) to HALT at PC=0x0006, showing ACC=2A (42), B=00, P=1, Pipeline trace MUL AB [0xA4], and 256B Internal RAM.*



Figure 3: *Cropped 3GPP LTE-B CRC-24b Lab on Web Dashboard verifying "123456789" -> Remainder 0x23EF52 (R5:23 R6:EF R7:52), Clean Syndrome 0x000000, 1-Bit Error (Byte 0 Bit 0: 0x31->0x30) 0x520000, and 2-Bit Adjacent Burst (Byte 0 Bits 0&1: 0x31->0x32) 0xF60000.*

4. Individual Contribution

4.1 Weekly Task Breakdown by Member

Table 4: *Weekly Task Allocation Across All Four Team Members*

Sprint Week	Cyrus Shobith Pinto (Team Lead)	Shashidhara Poojari (UI & Visuals)	Chrisel Lobo (Core & FDE)	Arnold Noah Karyappa (Logger & Telemetry)
Week 1	Project setup, architecture, Git repo, task allocation	Web tech research, HTML layout wireframes	8051 register & memory mapping research	Logger requirements & file appender design
Week 2	Opcode specification, ALU design, build tooling	CLI terminal display & status monitor prototype	CPU FDE cycle engine, arithmetic/logic opcodes	Log format design & log category structure
Week 3	Data structure specifications, assembly test plan	Stack & Circular FIFO visualizer UI layout	DataStructures.java: SP stack & 16-slot queue	Thread-safe synchronized writeLog implementation
Week 4	Multi-process socket IPC & non-blocking queue	UIProcess.java HTTP web server & preset buttons	CoreProcess.java socket server & state serializer	LoggerProcess.java socket listener & file writer
Week 5	CRC-24b step8051Hardware(), benchmarks, report	3-Stage CRC-24b Lab UI & real-time noise controls	Registers R5:R6:R7 mapping, core report sections	LoggerProcessTest.java backpressure & log audit

.

4.2 Agreed Team Contribution Summary

Table 5: *Agreed Team Member Workstream Distribution and Sign-Off*

Member Name	Student ID / USN	Assigned Workstreams	Confirmed Share (%)	Member Sign-Off
Cyrus Shobith Pinto	25190111	Architecture, Socket IPC, CRC-24b Engine, Benchmarking, Integration	25%	_____
Shashidhara Poojari	25190148	UIProcess HTTP Server, Responsive Dashboard, Visualizers, Presets	25%	_____
Chrisel Lobo	25190110	CoreProcess CPU Engine, ALU Opcodes, Registers, RAM, Stack, Docs	25%	_____
Arnold Noah Karyappa	25190106	LoggerProcess Daemon, LogHandler, Audit Persistence, Backpressure Tests	25%	_____

4.3 Individual Personal Reflections

Reflection by Cyrus Shobith Pinto (Team Leader - 25190111)

Leading the team through the design and integration of this multi-process simulator provided hands-on systems engineering experience. My primary technical focus was architecting the TCP socket IPC layer with a bounded non-blocking ArrayBlockingQueue(1024) so a stalled logger cannot block the CPU thread, and implementing the 3GPP TS 36.212 CRC-24b generator polynomial (g(D) = 0x800063) [5] via step8051Hardware(), shifting bit-by-bit through the carry flag across 8-bit registers R5, R6, and R7 in GF(2) arithmetic.

Reflection by Shashidhara Poojari (UI Subsystem Lead - 25190148)

Developing the Web Dashboard on Port 8080 required bridging low-level microcontroller state with an interactive browser interface. Designing the live mnemonic disassembly table to track the Program Counter (PC), alongside the 3-stage CRC-24b noise injection controls communicating over the /api/cmd HTTP query endpoint, gave me practical experience in client-server protocol handling and hardware visualization.

Reflection by Chrisel Lobo (Core Process Lead - 25190110)

Implementing the 8051 Fetch-Decode-Execute pipeline in Java required strict adherence to silicon constraints [2]. I learned how hardware flags like Carry (CY) and Overflow (OV) must be calculated to model physical arithmetic quirks, such as divide-by-zero setting OV=1 without crashing, and supporting both synthetic 0xFF HALT and standard 8051 SJMP $ (80 FE) self-loop halting.

Reflection by Arnold Noah Karyappa (Logging Subsystem Lead - 25190106)

Building LoggerProcess as an independent daemon on Port 5001 taught me the importance of process isolation and concurrency control. Using synchronized write blocks in LogHandler alongside our backpressure test suite (LoggerProcessTest TC-06) ensured that high-speed socket logging from CoreProcess never created interleaved log lines or stalled CPU execution.

5. Team Communication

Collaboration was maintained strictly using standardized agile and GitHub development practices:

- GitHub Pull Requests (PRs) with peer review mandates before merging sprint branches (`week-01` through `week-05` and `cyrus-integration`) into `main`.
- Detailed weekly meeting minutes following standard templates (agenda, technical decisions, participant action items) recorded in `Team_Bondaas_Meeting_Record.md`.
- Issue tracking and branch reconciliation verification via automated regression suites (`TestCPU`, `TestCRC24b`, `QueueTest`, `LoggerProcessTest`).

6. Problems Faced and How They Were Resolved

Table 6: *Engineering Issues Encountered, Root Causes, and Implemented Resolutions*

Issue Reference	Category	Symptom Description	Root Cause	Implemented Resolution
ISSUE-01	Java Datatypes	32-bit signed integer sign extension corrupted 8-bit register values	Java lacks unsigned byte type (0xFE treated as -2)	Applied bitwise mask (& 0xFF) across all ALU operations and register/RAM reads.
ISSUE-02	Silicon Memory	Stack underflow & stale ROM bytes after program reload	SP reset base at 07H [2]; loadProgram needed clean fill	Guarded pop() at SP <= 0x07 (fix(stack) commit) and zero-filled codeMemory on load.
ISSUE-03	Concurrency	Synchronous socket/disk logging risked stalling Core CPU if Logger stalled	Blocking TCP socket write when OS buffer fills	Added bounded ArrayBlockingQueue(1024) with offer(), drop counter, and daemon drain thread.
ISSUE-04	Algorithm	24-bit 3GPP LTE CRC remainder could not fit in 8-bit registers	STC89C52 registers are only 8 bits wide	Implemented step8051Hardware() shifting through carry bit across R5:R6:R7 in GF(2) XOR division.
ISSUE-05	Fault Handling	DIV AB instruction threw ArithmeticException on divide by zero	Unhandled software division by B=0	Implemented 8051 hardware spec [2]: B=0 sets Overflow (OV=1) without crashing.
ISSUE-06	Web Protocol	Hex bytecode with spaces failed to load from URL query string	HTTP URL encoding of spaces and symbols	Integrated URLDecoder.decode() in UIProcess /api/cmd HTTP request handler.

7. From Theory to Practice

7.1 Operating Systems Concepts

The project applies operating systems principles through process creation, inter-process communication (IPC) via loopback sockets, and bounded producer-consumer buffering. Decoupling the system into discrete OS processes (CoreProcess, LoggerProcess, UIProcess) with thread-safe monitor locks (cpuLock, synchronized LogHandler.writeLog) and a non-blocking 1,024-slot ArrayBlockingQueue ensures that a crash or stall in the UI or disk logger cannot corrupt or block CPU instruction execution.

7.2 Data Structures & Algorithms

Data structures were implemented to mirror physical hardware components [4]: a 16-slot circular FIFO buffer with head and tail wrap-around pointers ((rear + 1) % 16) for packet queuing, a Last-In First-Out (LIFO) hardware stack starting at SP=07H [2], and GF(2) linear feedback shift register division across R5:R6:R7 for CRC-24b calculation.

7.3 Computer Networks

Computer networking principles govern both the internal socket IPC wire framing and the telecommunications verification lab. The 3GPP TS 36.212 CRC-24b polynomial (g(D) = 0x800063) [5] demonstrates transport block error detection over noisy channels: for payload "123456789" (codeword 31..39 23 EF 52), clean reception yields syndrome 0x000000, flipping bit 0 of byte 0 ('1' 0x31->0x30) yields syndrome 0x520000, and flipping adjacent bits 0 and 1 of byte 0 (0x31->0x32) yields burst syndrome 0xF60000.

7.4 Cross-Disciplinary Synthesis Reflection

Developing this simulator demonstrated how core computer science disciplines intersect in systems software. Effective virtualization requires balancing computer architecture (instruction-accurate state machines [2], [4]) with operating systems principles (isolated processes, bounded non-blocking queues, synchronization) and networking techniques (TCP framing and GF(2) cyclic error detection [5]).

8. Conclusion and Future Work

This project successfully engineered, integrated, and verified an educational multi-process simulator for a 17-opcode subset of the STC89C52 (8051) microcontroller [2], [3]. The delivered system features an instruction-accurate Fetch-Decode-Execute CPU engine supporting both synthetic 0xFF HALT and standard 8051 SJMP $ (80 FE) halting, 8 KB code memory, 256-byte RAM and SFR management, a 16-slot circular FIFO queue, an asynchronous append-only TCP audit logger on Port 5001 backed by a non-blocking 1,024-slot queue, a Web Dashboard on Port 8080, and a verified 3GPP TS 36.212 CRC-24b telecom lab [5]. Automated regression suites confirm 34/34 passing assertions across TestCPU (17/17), QueueTest (3/3), LoggerProcessTest (6/6), and TestCRC24b (8/8). Future work includes expanding the opcode decoder to the full 255-instruction MCS-51 table with machine-cycle timing to execute compiled Keil/SDCC Intel HEX binaries, and adding CH340 USB-UART hardware-in-the-loop bridging.

References

[1] Department of Computer Science & Engineering, "PBL Simulator Evaluation Rubrics & Task Sheets," St. Aloysius (Deemed to be University), Mangaluru, 2026.

[2] Intel Corporation, "MCS-51 Microcontroller Family User's Manual," Publication No. 272383-002, 1994; and STC Micro, "STC89C52RC/54RD+ Series 8-Bit Microcontroller Datasheet," 2011.

[3] P. C. Vijay Ganesh, "8051 Microcontroller Web-based Simulator for Academic Instruction," GitHub Repository, 2021.

[4] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed. Morgan Kaufmann, 2014.

[5] 3GPP TS 36.212, "Evolved Universal Terrestrial Radio Access (E-UTRA); Multiplexing and channel coding (Release 15)," Section 5.1.1 (CRC-24b generator polynomial gCRC24B(D)), 2018.

Final Report Sign-off

By signing below, the team members confirm that the documented contribution distributions, source code statistics, and peer evaluations represent an accurate record of the project lifecycle.

Team Leader Signature: _____ Date: 07 November 2026

Faculty Mentor Endorsement:

I have reviewed this Project-Based Learning Final Report along with the associated repository, code contributions, and test suite.

Mentor Signature: _____ Date: _____